

Semaphores are integer variables that are used to solve the critical section problem by using two atomic operations, wait and signal that are used for process synchronization.

The definitions of wait and signal are as follows –

- **Wait**

The wait operation decrements the value of its argument **S**, if it is positive. If **S** is negative or zero, then no operation is performed.

```
wait(S)
{
    while (S<=0);

    S--;
}
```

- **Signal**

The signal operation increments the value of its argument **S**.

```
signal(S)
{
    S++;
}
```

Types of Semaphores

There are two main types of semaphores i.e. counting semaphores and binary semaphores. Details about these are given as follows –

- **Counting Semaphores**

These are integer value semaphores and have an unrestricted value domain. These semaphores are used to coordinate the resource access, where the semaphore count is the number of available resources. If the resources are added, semaphore count automatically incremented and if the resources are removed, the count is decremented.

- **Binary Semaphores**

The binary semaphores are like counting semaphores but their value is restricted to 0 and 1. The wait operation only works when the semaphore is 1 and the signal operation succeeds when semaphore is 0. It is sometimes easier to implement binary semaphores than counting semaphores.



Advantages of Semaphores

Some of the advantages of semaphores are as follows –

- Semaphores allow only one process into the critical section. They follow the mutual exclusion principle strictly and are much more efficient than some other methods of synchronization.
- There is no resource wastage because of busy waiting in semaphores as processor time is not wasted unnecessarily to check if a condition is fulfilled to allow a process to access the critical section.
- Semaphores are implemented in the machine independent code of the microkernel. So they are machine independent.

Disadvantages of Semaphores

Some of the disadvantages of semaphores are as follows –

- Semaphores are complicated so the wait and signal operations must be implemented in the correct order to prevent deadlocks.
- Semaphores are impractical for last scale use as their use leads to loss of modularity. This happens because the wait and signal operations prevent the creation of a structured layout for the system.
- Semaphores may lead to a priority inversion where low priority processes may access the critical section first and high priority processes later.

What is the Producer-Consumer Problem?

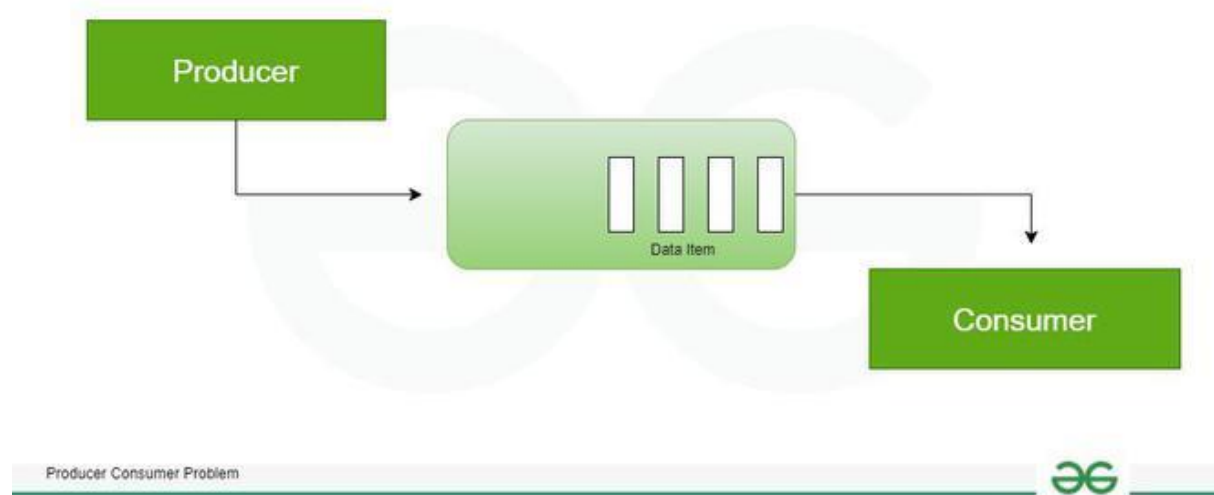
The producer-consumer problem is an example of a [multi-process synchronization](#) problem. The problem describes two processes, the producer and the consumer that share a common fixed-size buffer and use it as a [queue](#).

- The producer's job is to generate data, put it into the buffer, and start again.
- At the same time, the consumer is consuming the data (i.e., removing it from the buffer), one piece at a time.

What is the Actual Problem?

Given the common fixed-size buffer, the task is to make sure that the producer can't add data into the buffer when it is full and the consumer can't remove data from an

empty buffer. Accessing memory buffers should not be allowed to producers and consumers at the same time.



Below are a few points that considered as the problems occur in Producer-Consumer:

- The producer should produce data only when the buffer is not full. In case it is found that the buffer is full, the producer is not allowed to store any data into the memory buffer.
- Data can only be consumed by the consumer if and only if the memory buffer is not empty. In case it is found that the buffer is empty, the consumer is not allowed to use any data from the memory buffer.
- Accessing memory buffer should not be allowed to producer and consumer at the same time.

Let's see the code for the above problem:

Producer Code

```

int count = 0;
void producer( void )
{
    int itemP;
    while(1)
    {
        Produce_item(item P)
        while( count == n);
        buffer[ in ] = item P;
        in = (in + 1)mod n
        count = count + 1;
    }
}

```

Load Rp, m[count]
Increment Rp
Store m[count], Rp

Producer Code

```

int count;
void consumer(void)
{
    int itemC;
    while( 1 )
    {
        while( count == 0 );
        itemC = buffer[ out ];
        out = ( out + 1 ) mod n;
        count = count - 1;
    }
}

```

Load Rc, m[count]
Decrement Rc
Store m[count], Rc

Let's understand above Producer and Consumer code:

1. "in" used in a producer code represent the next **empty buffer**
2. "out" used in consumer code represent first **filled buffer**
3. count keeps the count number of elements in the buffer
4. count is further divided into 3 lines code represented in the block in both the producer and consumer code.

If we talk about Producer code first:

- Rp is a register which keeps the value of m[count]
- Rp is incremented (As element has been added to buffer)
- an Incremented value of Rp is stored back to m[count]

READERS WRITERS PROBLEM

The readers-writers problem is a classical problem of process synchronization, it relates to a data set such as a file that is shared between more than one process at a time. Among these various processes, some are Readers - which can only read the data set; they do not perform any updates, some are Writers - can both read and write in the data sets.

The readers-writers problem is used for managing synchronization among various reader and writer process so that there are no problems with the data sets, i.e. no inconsistency is generated.

Let's understand with an example - If two or more than two readers want to access the file at the same point in time there will be no problem. However, in other situations like when two writers or one reader and one writer wants to access the file at the same point of time, there may occur some problems, hence the task is to design the code in such a manner that if one reader is reading then no writer is allowed to update at the same point of time, similarly, if one writer is writing no reader is allowed to read the file at that point of time and if one writer is updating a file other writers should not be allowed to update the file at the same point of time. However, multiple readers can access the object at the same time.

Let us understand the possibility of reading and writing with the table given below:

TABLE 1

Case	Process 1	Process 2	Allowed / Not Allowed
Case 1	Writing	Writing	Not Allowed
Case 2	Reading	Writing	Not Allowed
Case 3	Writing	Reading	Not Allowed
Case 4	Reading	Reading	Allowed

The solution of readers and writers can be implemented using binary semaphores.

We use two binary semaphores "write" and "mutex", where binary semaphore can be defined as:

Semaphore: A semaphore is an integer variable in S, that apart from initialization is accessed by only two standard atomic operations - wait and signal, whose definitions are as follows:

1. 1. wait(S)
2. {
3. **while**(S <= 0) ;
4. S--;
5. }
- 6.
7. 2. signal(S)
8. {
9. S++;
- 10.}

From the above definitions of wait, it is clear that if the value of $S \leq 0$ then it will enter into an infinite loop (because of the semicolon; after while loop). Whereas the job of the signal is to increment the value of S.

The below code will provide the solution of the reader-writer problem, reader and writer process codes are given as follows -

Code for Reader Process

The code of the reader process is given below -

1. **static int** readcount = 0;
2. wait (mutex);
3. readcount ++; // on each entry of reader increment readcount
4. **if** (readcount == 1)
5. {
6. wait (write);
7. }
8. signal(mutex);
- 9.
- 10.--READ THE **FILE**?
- 11.
- 12.wait(mutex);
- 13.readcount --; // on every exit of reader decrement readcount

```
14. if (readcount == 0)
15. {
16.  signal (write);
17. }
18. signal(mutex);
```

In the above code of reader, **mutex** and **write** are **semaphores** that have an initial value of 1, whereas **the readcount** variable has an initial value as 0.

Both **mutex** and **write** are common in reader and writer process code, semaphore **mutex** ensures mutual exclusion and semaphore **write** handles the writing mechanism.

The **readcount** variable denotes the number of readers accessing the file concurrently. The moment variable **readcount** becomes 1, **wait** operation is used to write semaphore which decreases the value by one. This means that a writer is not allowed how to access the file anymore. On completion of the read operation, **readcount** is decremented by one. When **readcount** becomes 0, the signal operation which is used to **write** permits a writer to access the file.

Code for Writer Process

The code that defines the writer process is given below:

```
1. wait(write);
2. WRITE INTO THE FILE
3. signal(wrt);
```

If a writer wishes to access the file, **wait** operation is performed on **write** semaphore, which decrements **write** to 0 and no other writer can access the file. On completion of the writing job by the writer who was accessing the file, the signal operation is performed on **write**.

Dining Philosophers problem

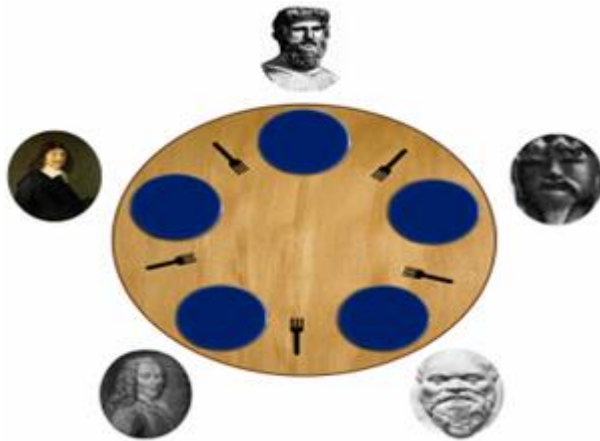
Overview :

[Dining Philosophers Problem](#) States that there are 5 Philosophers who are engaged in two activities Thinking and Eating. Meals are taken communally in a table with five plates and five forks in a cyclic manner as shown in the figure.

Constraints and Condition for the problem :

1. Every Philosopher needs two forks in order to eat.

2. Every Philosopher may pick up the forks on the left or right but only one fork at once.
3. Philosophers only eat when they had two forks. We have to design such a protocol i.e. pre and post protocol which ensures that a philosopher only eats if he or she had two forks.
4. Each fork is either clean or dirty.



Solution :

Correctness properties it needs to satisfy are :

1. **Mutual Exclusion Principle –**
No two Philosophers can have the two forks simultaneously.
2. **Free from Deadlock –**
Each philosopher can get the chance to eat in a certain finite time.
3. **Free from Starvation –**When few Philosophers are waiting then one gets a chance to eat in a while.
4. No strict Alternation.
5. Proper utilization of time.

Algorithm(outline) :

```

loop forever
  p1: think
  p2: preprotocol
  p3: eat
  p4: postprotocol

```

First Attempt :

We assume that each philosopher is initialized with its index I and that addition is implicitly modulo 5. Each fork is modeled as a semaphore where wait corresponds to taking a fork and signal corresponds to putting down a fork.

Algorithm –

```
semaphore array[0..4] fork ← [1, 1, 1, 1, 1]
loop forever
    p1 : think
    p2 : wait(fork[i])
    p3 : wait(fork[i + 1])
    p4 : eat
    p5 : signal(fork[i])
    p6 : signal(fork[i + 1])
```

Problem with this solution :

This solution may lead to a deadlock under an interleaving that has all the philosophers pick up their left forks before any of them tries to pick up a right fork. In this case, all the Philosophers are waiting for the right fork but no one will execute a single instruction.

Second Attempt :

One way to tackle the above situation is to limit the number of philosophers entering the room to four. By doing this one of the philosophers will eventually get both the fork and execute all the instruction leading to no deadlock.

Algorithm –

```
semaphore array[0..4] fork ← [1, 1, 1, 1, 1]

semaphore room ← 4
loop forever
    p1 : think
    p2 : wait(room)
    p3 : wait(fork[i])
    p4 : wait(fork[i + 1])
    p5 : eat
    p6 : signal(fork[i])
    p7 : signal(fork[i + 1])
    p8 : signal(room)
```

In this solution, we somehow interfere with the given problem as we allow only four philosophers.